

دانشگاه تهران

پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر

تمرین شماره: ۱

شبکه‌های عصبی و یادگیری عمیق

نام و نام خانوادگی: آرتین توسلی

شماره دانشجویی: ۸۱۰۱۰۲۵۴۳

نیم‌سال دوم

سال تحصیلی ۴۰۴-۴۰۵

فهرست مطالب

۴	۱	Autoencoder+Classifier
۴	۱.۱	تئوری
۴	۱.۱.۱	MADALINE با XOR
۷	۲.۱.۱	قانون دلتا
۱۰	۲.۱	پیاده سازی
۱۰	۱.۲.۱	بارگذاری داده ها و تابع پیش پردازش
۱۱	۲.۲.۱	پیاده سازی اتوانکدرها
۱۲	۳.۲.۱	آموزش مدل اتوانکدر
۱۲	۴.۲.۱	پیاده سازی شبکه طبقه بند
۱۳	۵.۲.۱	ارزیابی دو نوع طبقه بند
۱۵	۲	Regression
۱۵	۱.۲	تئوری
۱۵	۱.۱.۲	MLP
۱۶	۲.۱.۲	Backpropagation
۱۸	۲.۲	پیاده سازی
۱۸	۱.۲.۲	پیش پردازش داده ها
۱۹	۲.۲.۲	آموزش شبکه عصبی ساده
۲۱	۳.۲.۲	آموزش شبکه عصبی پیشرفته
۲۲	۴.۲.۲	تحلیل نتایج

فهرست تصاویر

۱۰.۱	نواحی تصمیم‌گیری با شیب مثبت	۷
۲۰.۱	توزیع داده‌های آموزش	۱۱
۳۰.۱	توزیع داده‌های تست	۱۱
۴۰.۱	نمودار خطای آموزش (MSE) برای اتوانکدرهای V۱ و V۲ در طول ایپاک‌ها	۱۳
۵۰.۱	نمودارهای دقت و خطای تست برای طبقه‌بندهای V۱ و V۲	۱۴
۱۰.۲	توزیع بردارهای ویژگی دو کلاس ω_1 و ω_2	۱۶
۲۰.۲	ساختار شبکه عصبی MLP ($2 \rightarrow 2 \rightarrow 1$)	۱۶
۳۰.۲	ساختار شبکه عصبی دولایه با n_x ویژگی و n_h نورون پنهان	۱۷
۴۰.۲	تغییرات تابع هزینه (Loss) در طول ۲۰۰ ایپاک برای مدل‌های اول و دوم	۲۱
۵۰.۲	تغییرات تابع هزینه (Loss) در طول آموزش برای مدل پیشرفته (مدل سوم)	۲۳

فهرست جداول

۱۲		۱۰.۱ معماری و هایپرپارامترهای شبکه‌های اتوانکدر
۱۳		۲۰.۱ معماری بخش طبقه‌بند در شبکه‌های پیش‌خور
۲۴		۱۰.۲ مقایسه عملکرد سه معماری مختلف شبکه عصبی

سوال ۱

Autoencoder+Classifier

۱.۱ تئوری

۱.۱.۱ XOR با MADALINE

تابع منطقی XOR بر اساس فرمول ارائه شده در صورت سوال به شکل زیر تجزیه می‌شود:

$$X_1 \oplus X_2 = (X_1 \vee X_2) \wedge \neg(X_1 \wedge X_2)$$

این عبارت نشان می‌دهد که خروجی XOR برابر است با AND کردن خروجی یک گیت OR و یک گیت NAND. در یک شبکه، MADALINE این ساختار با دو لایه پیاده‌سازی می‌شود:

- لایه مخفی: شامل دو نورون (Adaline) است. نورون اول (Z_1) ناحیه مربوط به گیت OR را جدا می‌کند و نورون دوم (Z_2) ناحیه مربوط به گیت NAND را.
- لایه خروجی: یک نورون است که عملگر AND را روی خروجی‌های Z_1 و Z_2 اعمال می‌کند.

فرض می‌کنیم ورودی‌ها باینری باشند ($X_1, X_2 \in \{0, 1\}$).

- نورون Z_1 (گیت OR): خط تصمیم باید نقطه $(0, 0)$ را از سه نقطه دیگر جدا کند. معادله خط

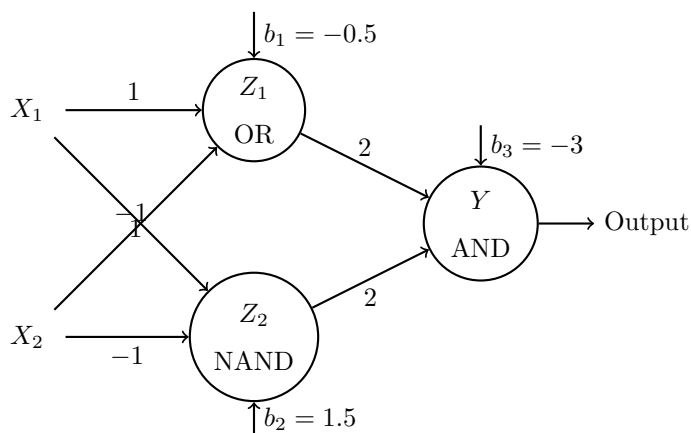
می‌تواند $x_2 = 0.5 - x_1$ باشد. بنابراین وزن‌ها $w_{11} = 1, w_{21} = 1$ و بایاس $b_1 = -0.5$ است.

• نورون Z_2 (گیت NAND): خط تصمیم باید نقطه $(1, 1)$ را از سه نقطه دیگر جدا کند. معادله خط می‌تواند $x_2 = 1.5 - x_1$ باشد. از آنجا که ناحیه مثبت زیر این خط است، پس: $-x_1 - x_2 + 1.5 > 0$. بنابراین وزن‌ها $w_{12} = -1, w_{22} = -1$ و بایاس $b_2 = 1.5$ است.

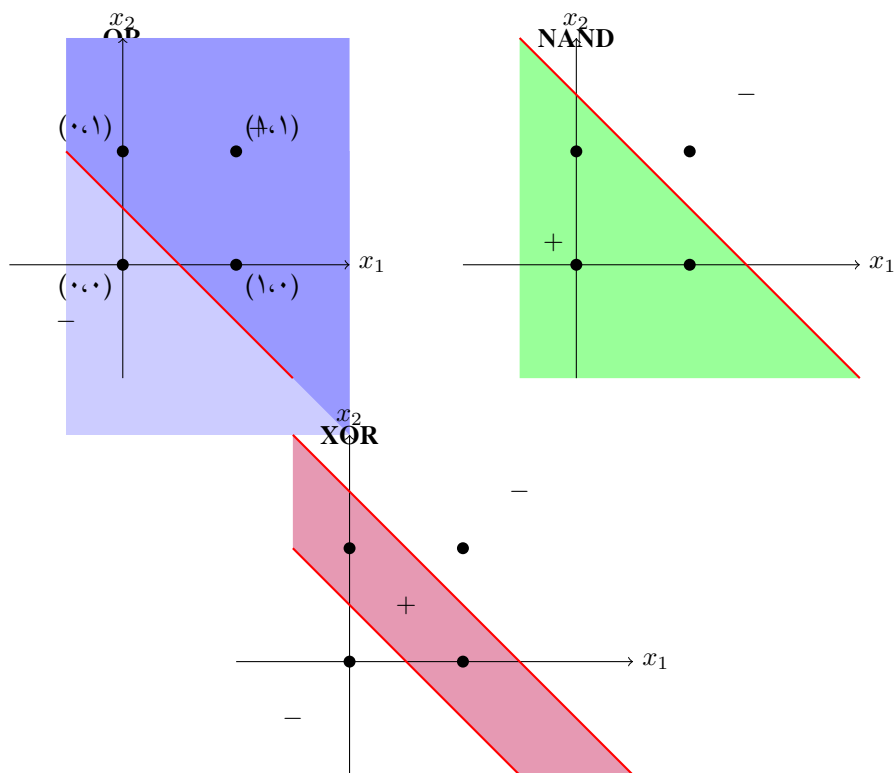
• نورون خروجی Y (گیت AND): باید زمانی خروجی ۱ بدهد که هر دو ورودی Z_1 و Z_2 برابر ۱ باشند. به عنوان یک ترکیب جایگزین، فرض می‌کنیم وزن‌ها $v_1 = 2, v_2 = 2$ و بایاس $b_3 = -3$ باشند.

در این حالت مجموع ورودی‌ها به شکل $Net = 2Z_1 + 2Z_2 - 3$ محاسبه می‌شود. تنها در صورتی که هر دو ورودی $Z_1 = 1$ و $Z_2 = 1$ باشند، مقدار $Net = 4 - 3 = 1$ می‌شود که بزرگتر از صفر است و خروجی نورون فعال می‌شود. (در هر حالت دیگر نورون خاموش می‌باشد دقیقاً مانند گیت AND)

ساختار شبکه:



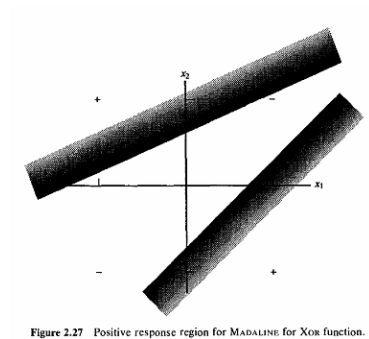
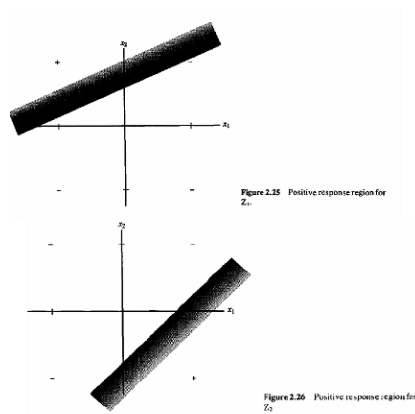
نواحی پاسخ شبکه:



در نمودار نهایی، مشاهده می‌شود که ناحیه اشتراک (بخش بنفش‌رنگ)، یک نوار با شیب منفی است که دقیقاً نقاط $(0, 1)$ و $(1, 0)$ که جواب‌های مثبت XOR هستند را در بر می‌گیرد و نقاط $(0, 0)$ و $(1, 1)$ خارج از این نوار (در نواحی منفی) قرار می‌گیرند.

مقایسه با مثال ۲۱.۲ کتاب مرجع

در هر دو روش، چه در قسمت الف و چه در مثال کتاب، لایه خروجی دقیقاً مثل یک گیت AND عمل می‌کند و کارش این است که اشتراک نواحی ایجاد شده توسط لایه مخفی را به دست بیاورد. تفاوت اصلی این دو روش به نحوه تقسیم‌بندی فضا توسط لایه مخفی برمی‌گردد. در روشی که ما استفاده کردیم، دو نوار لایه مخفی دو خط موازی با شیب منفی رسم می‌کنند. وقتی لایه خروجی اشتراک این دو ناحیه را می‌گیرد، یک نوار مورب با شیب منفی بین این دو خط به دست می‌آید. اما در مثال کتاب، از دو خط با شیب مثبت استفاده شده است. این خطوط فضا را طوری تقسیم می‌کنند که وقتی لایه خروجی اشتراک آن‌ها را محاسبه می‌کند، ناحیه جواب به شکل دو بخش مجزا درمی‌آید. در واقع هر دو شبکه در نهایت تابع XOR را درست حل می‌کنند، اما از نظر هندسی از دو راه کاملاً متفاوت برای جدا کردن ورودی‌ها استفاده کرده‌اند.



شکل ۱.۱: نواحی تصمیم‌گیری با شیب مثبت

محدودیت یک پرسپترون منفرد

یک پرسپترون منفرد تنها قادر به یادگیری و دسته‌بندی الگوهایی است که به‌طور خطی تفکیک‌پذیر باشند. این بدان معناست که پرسپترون خطی، تنها می‌تواند یک خط مستقیم (یا یک ابرصفحه در ابعاد بالاتر) را در فضای ویژگی‌ها رسم کند تا کلاس‌ها را از هم جدا کند. همانطور که از شکل XOR در بالا معلوم است، هیچ خط مستقیمی وجود ندارد که مثبت‌ها را از منفی‌ها جدا کند و بخاطر همین با یک لایه پرسپترون نمیتوانیم مدل‌اش کنیم. با اضافه کردن یک لایه پنهان میتوانیم چندین تصمیم‌گیر خطی را ترکیب کنیم و XOR را پیاده‌سازی کنیم همانطور که نشان داده شد.

۲.۱.۱ قانون دلتا

بردارهای ورودی را با اضافه کردن ورودی بایاس ($x_0 = 1$) به صورت زیر تعریف می‌کنیم:

$$\begin{aligned} X_1 &= [1, 1, 1, 1, 1]^T, & t_1 &= 1 \\ X_2 &= [1, -1, 1, -1, -1]^T, & t_2 &= 1 \\ X_3 &= [1, 1, 1, 1, -1]^T, & t_3 &= -1 \\ X_4 &= [1, 1, -1, -1, 1]^T, & t_4 &= -1 \end{aligned}$$

بردار وزن‌های اولیه $W_1 = [0, 0, 0, 0, 0]^T$ و نرخ یادگیری $\alpha = 0.5$ است. قانون به‌روزرسانی وزن‌ها:

$$\Delta W = \alpha(t - y)X \implies W_{new} = W_{old} + \Delta W$$

و $net = W^T X$ و تابع فعال‌سازی $y = \text{sgn}(net)$

ایپاک اول

• ورودی X_1 :

$$net = W^T X_1 = [0, 0, 0, 0, 0] \cdot [1, 1, 1, 1, 1]^T = 0 \implies y = 1$$

$$t_1 = 1 \implies \text{Error} = 1 - 1 = 0 \implies \Delta W = 0$$

$$W = [0, 0, 0, 0, 0]^T$$

• ورودی X_2 :

$$net = W^T X_2 = [0, 0, 0, 0, 0] \cdot [1, -1, 1, -1, -1]^T = 0 \implies y = 1$$

$$t_2 = 1 \implies \text{Error} = 1 - 1 = 0 \implies \Delta W = 0$$

$$W = [0, 0, 0, 0, 0]^T$$

• ورودی X_3 :

$$net = W^T X_3 = [0, 0, 0, 0, 0] \cdot [1, 1, 1, 1, -1]^T = 0 \implies y = 1$$

$$t_3 = -1 \implies \text{Error} = -1 - 1 = -2$$

$$\Delta W = 0.5(-2)[1, 1, 1, 1, -1]^T = [-1, -1, -1, -1, 1]^T$$

$$W = W + \Delta W = [-1, -1, -1, -1, 1]^T$$

• ورودی X_4 :

$$net = W^T X_4 = [-1, -1, -1, -1, 1] \cdot [1, 1, -1, -1, 1]^T = -1 - 1 + 1 + 1 + 1 =$$

$$1 \implies y = 1$$

$$t_4 = -1 \implies \text{Error} = -1 - 1 = -2$$

$$\Delta W = 0.5(-2)[1, 1, -1, -1, 1]^T = [-1, -1, 1, 1, -1]^T$$

$$W = [-1, -1, -1, -1, 1]^T + [-1, -1, 1, 1, -1]^T = [-2, -2, 0, 0, 0]^T$$

ایپاک دوم

• ورودی X_1 :

$$\begin{aligned}
 net &= [-2, -2, 0, 0, 0] \cdot [1, 1, 1, 1, 1]^T = -4 \implies y = -1 \\
 \text{Error} &= 1 - (-1) = 2 \implies \Delta W = 0.5(2)[1, 1, 1, 1, 1]^T = [1, 1, 1, 1, 1]^T \\
 W &= [-2, -2, 0, 0, 0]^T + [1, 1, 1, 1, 1]^T = [-1, -1, 1, 1, 1]^T
 \end{aligned}$$

• ورودی X_2 :

$$\begin{aligned}
 net &= [-1, -1, 1, 1, 1] \cdot [1, -1, 1, -1, -1]^T = -1 + 1 + 1 - 1 - 1 = -1 \implies \\
 & y = -1 \\
 \text{Error} &= 1 - (-1) = 2 \implies \Delta W = 0.5(2)[1, -1, 1, -1, -1]^T = [1, -1, 1, -1, -1]^T \\
 W &= [-1, -1, 1, 1, 1]^T + [1, -1, 1, -1, -1]^T = [0, -2, 2, 0, 0]^T
 \end{aligned}$$

• ورودی X_3 :

$$\begin{aligned}
 net &= [0, -2, 2, 0, 0] \cdot [1, 1, 1, 1, -1]^T = 0 - 2 + 2 + 0 + 0 = 0 \implies y = 1 \\
 \text{Error} &= -1 - 1 = -2 \implies \Delta W = 0.5(-2)[1, 1, 1, 1, -1]^T = [-1, -1, -1, -1, 1]^T \\
 W &= [0, -2, 2, 0, 0]^T + [-1, -1, -1, -1, 1]^T = [-1, -3, 1, -1, 1]^T
 \end{aligned}$$

• ورودی X_4 :

$$\begin{aligned}
 net &= [-1, -3, 1, -1, 1] \cdot [1, 1, -1, -1, 1]^T = -1 - 3 - 1 + 1 + 1 = -3 \implies \\
 & y = -1 \\
 \text{Error} &= -1 - (-1) = 0 \implies \Delta W = 0 \\
 W &= [-1, -3, 1, -1, 1]^T
 \end{aligned}$$

ایپاک سوم

• ورودی X_1 :

$$\begin{aligned}
 net &= [-1, -3, 1, -1, 1] \cdot [1, 1, 1, 1, 1]^T = -3 \implies y = -1 \\
 \text{Error} &= 1 - (-1) = 2 \implies \Delta W = 0.5(2)[1, 1, 1, 1, 1]^T = [1, 1, 1, 1, 1]^T \\
 W &= [-1, -3, 1, -1, 1]^T + [1, 1, 1, 1, 1]^T = [0, -2, 2, 0, 2]^T
 \end{aligned}$$

• ورودی X_2 :

$$\begin{aligned}
 net &= [0, -2, 2, 0, 2] \cdot [1, -1, 1, -1, -1]^T = 0 + 2 + 2 + 0 - 2 = 2 \implies y = 1 \\
 \text{Error} &= 1 - 1 = 0 \implies \Delta W = 0 \implies W = [0, -2, 2, 0, 2]^T
 \end{aligned}$$

• ورودی X_3 :

$$net = [0, -2, 2, 0, 2] \cdot [1, 1, 1, 1, -1]^T = 0 - 2 + 2 + 0 - 2 = -2 \implies y = -1$$

$$Error = -1 - (-1) = 0 \implies \Delta W = 0 \implies W = [0, -2, 2, 0, 2]^T$$

• ورودی X_4 :

$$net = [0, -2, 2, 0, 2] \cdot [1, 1, -1, -1, 1]^T = 0 - 2 - 2 + 0 + 2 = -2 \implies y = -1$$

$$Error = -1 - (-1) = 0 \implies \Delta W = 0 \implies W = [0, -2, 2, 0, 2]^T$$

ایپاک چهارم

با اعمال وزن‌های به دست آمده $W = [0, -2, 2, 0, 2]^T$ به تمامی ورودی‌ها:

$$net_1 = W^T X_1 = 0 - 2 + 2 + 0 + 2 = 2 \implies y_1 = 1 = t_1$$

$$net_2 = W^T X_2 = 0 + 2 + 2 + 0 - 2 = 2 \implies y_2 = 1 = t_2$$

$$net_3 = W^T X_3 = 0 - 2 + 2 + 0 - 2 = -2 \implies y_3 = -1 = t_3$$

$$net_4 = W^T X_4 = 0 - 2 - 2 + 0 + 2 = -2 \implies y_4 = -1 = t_4$$

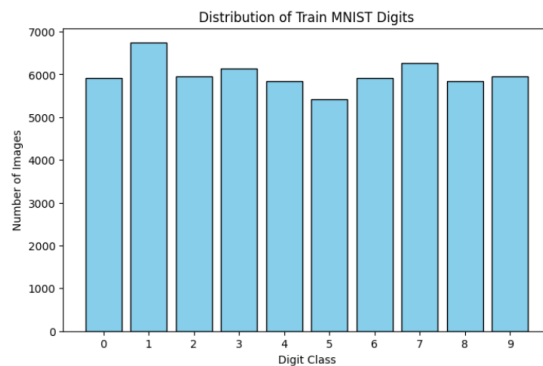
همان‌طور که مشاهده می‌شود، خطای تمامی داده‌ها صفر است. شبکه به همگرایی رسیده است و بردار وزن نهایی به همراه بایاس برابر است با:

$$W_{final} = [0, -2, 2, 0, 2]^T$$

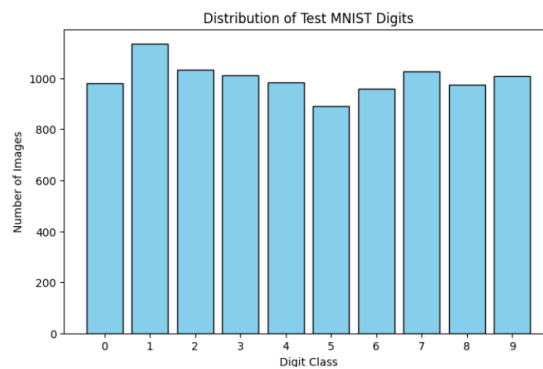
۲.۱ پیاده سازی

۱.۲.۱ بارگذاری داده‌ها و تابع پیش‌پردازش

در این مرحله، مجموعه داده‌ی MNIST، که شامل تصاویر ۲۸ در ۲۸ ارقام دست‌نویس است بارگذاری شد. داده‌ها به دسته‌های آموزش و تست تقسیم شدند (۶۰۰۰۰ داده آموزش و ۱۰۰۰۰ داده تست) نمودار توزیع هر یک از دسته‌های داده آموزش و تست در شکل ۲.۱ و ۳.۱ به ترتیب آمده است. در بخش پیش‌پردازش، مقادیر پیکسل‌ها که در بازه‌ی $[0, 255]$ قرار داشتند، بر عدد ۲۵۵.۰ تقسیم شدند تا نرمال‌سازی شده و در محدوده‌ی $[0, 1]$ قرار گیرند. سپس، ابعاد تصاویر از ماتریس‌های دوبعدی 28×28 به بردارهای یک‌بعدی با طول ۷۸۴ تبدیل شدند. (Flatten) داده‌های آموزش و تست پس از این پیش‌پردازش، به فرمت تانسورهای PyTorch



شکل ۲.۱: توزیع داده های آموزش



شکل ۳.۱: توزیع داده های تست

تبدیل شده و درون DataLoader قرار گرفتند تا برای Training Batch با Batch Size برابر با 64 آماده شوند.

۲.۲.۱ پیاده سازی اتوانکدرها

در این آزمایش، دو معماری مختلف برای شبکه اتوانکدر پیاده سازی شد. تفاوت اصلی این دو مدل در اندازه بعد فضای پنهان یا گلوگاه آن هاست. در جدول ۱.۱ معماری این دو را میتوانید ببینید. تحلیل تعداد پارامترها: فرمول محاسبه پارامترهای هر لایه خطی به صورت $(n_{in} \times n_{out}) + n_{out}$ می باشد. (چون شبکه fully connected هست هر نورون ورودی به هر نورون خروجی متصل میباشد و یک پارامتر دارد و همینطور عبارت دوم بخاطر این است که یک bias به همه نورون های خروجی وصل است). برای مدل V۱ (گلوگاه ۸):

مدل	بخش	لایه ها (ابعاد)	تابع فعال ساز
V۱	انکدر	$784 \rightarrow 128 \rightarrow 8$	ReLU
	دیکدر	$8 \rightarrow 128 \rightarrow 784$	ReLU
V۲	انکدر	$784 \rightarrow 128 \rightarrow 4$	ReLU
	دیکدر	$4 \rightarrow 128 \rightarrow 784$	ReLU

جدول ۱.۱: معماری و هایپرپارامترهای شبکه های اتوانکدر

- انکدر: $(784 \times 128 + 128) + (128 \times 8 + 8) = 100,480 + 1,032 = 101,512$
- دیکدر: $(8 \times 128 + 128) + (128 \times 784 + 784) = 1,152 + 101,136 = 102,288$
- مجموع: $101,512 + 102,288 = 203,800$ پارامتر.

برای مدل V۲ (گلگاه ۴):

- انکدر: $(784 \times 128 + 128) + (128 \times 4 + 4) = 100,480 + 516 = 100,996$
- دیکدر: $(4 \times 128 + 128) + (128 \times 784 + 784) = 640 + 101,136 = 101,776$
- مجموع: $100,996 + 101,776 = 202,772$ پارامتر.

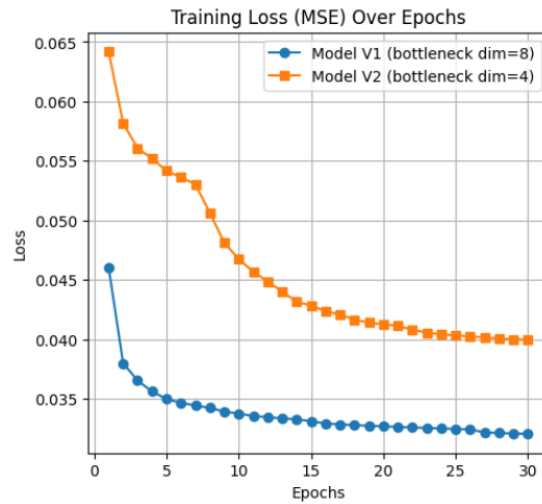
خروجی چاپ شده توسط کد نیز دقیقاً با این محاسبات تئوری مطابقت دارد.

۳.۲.۱ آموزش مدل اتوانکدر

مدل های اتوانکدر با استفاده از تابع هزینه میانگین مربعات خطا و بهینه ساز Adam با نرخ یادگیری برابر با 0.001 به مدت 30 اپیاک آموزش دیدند. همان طور که در شکل ۴.۱ مشاهده می شود، مدل V۱ (با بعد گلگاه ۸) خطای بسیار کمتری نسبت به مدل V۲ (با بعد گلگاه ۴) دارد. این نتیجه منطقی است چون که گلگاه بزرگتر به شبکه اجازه می دهد اطلاعات بیشتری از تصویر اصلی را در فضای پنهان ذخیره کند و در دیکدر نیز می تواند بازسازی دقیق تری انجام دهد ولی فشرده سازی بیش از حد در مدل V۲ باعث از دست رفتن بخش زیادی از اطلاعات شده است.

۴.۲.۱ پیاده سازی شبکه طبقه بند

حال می خواهیم از شبکه قسمت قبل کمک بگیریم تا اعداد را طبقه بندی کنیم. برای اینکار از تکنیک Transfer Learning استفاده کنیم. انکودر توانسته است عکس ها را از فضای با بعد زیاد به بعد کمتر برسد و یعنی توانسته است ویژگی های مهم عکس ها را خلاصه کند حال می خواهیم از خروجی همین استفاده کنیم بخاطر همین پارامتر



شکل ۴.۱: نمودار خطای آموزش (MSE) برای اتوانکدرهای V۱ و V۲ در طول ایپاک‌ها

های آن را فریز میکنیم تا دیگر آموزش نبیند. سپس یک شبکه عصبی Feed-forward ساده به عنوان Head طبقه‌بند به انتهای انکدرها اضافه شد:

مدل طبقه‌بند	لایه‌های متصل به انکدر (Trainable)	تابع فعال‌ساز
V۱ Classifier	$8 \rightarrow 4 \rightarrow 10$	ReLU در لایه اول
V۲ Classifier	$4 \rightarrow 10$	ندارد (مستقیم به خروجی)

جدول ۲.۱: معماری بخش طبقه‌بند در شبکه‌های پیش‌خور

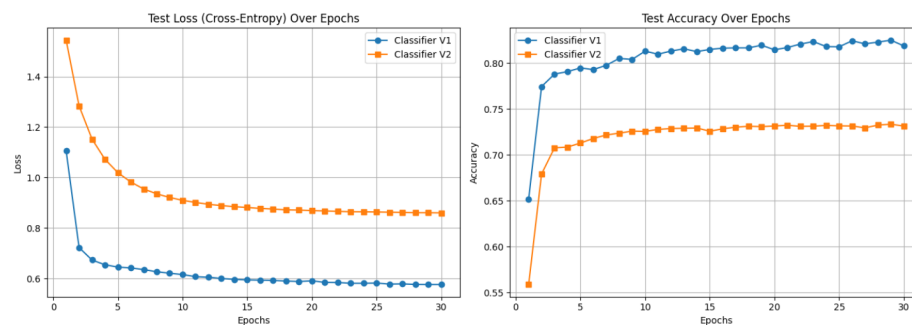
تحلیل پارامترها:

- طبقه‌بند V1: انکدر این مدل دارای 101,512 پارامتر است که همگی فریز شده‌اند. بخش طبقه‌بند جدید شامل دو لایه است: لایه اول $(8 \times 4 + 4) = 36$ و لایه دوم $(4 \times 10 + 10) = 50$ پارامتر دارد. در مجموع 86 پارامتر قابل آموزش (Unfrozen) وجود دارد. مجموع کل پارامترها 101,598 است.
- طبقه‌بند V2: انکدر این مدل دارای 100,996 پارامتر فریز شده است. لایه طبقه‌بند نهایی آن $(4 \times 10 + 10) = 50$ پارامتر قابل آموزش دارد. مجموع کل پارامترها 101,046 است.

خروجی چاپ شده توسط کد نیز دقیقاً با این محاسبات تئوری مطابقت دارد.

۵.۲.۱ ارزیابی دو نوع طبقه‌بند

مدل‌های طبقه‌بند با استفاده از تابع هزینه Cross-Entropy و بهینه‌ساز Adam (که تنها روی پارامترهای Unfrozen اعمال شد) برای 30 اپیک آموزش داده شدند.



شکل ۵.۱: نمودارهای دقت و خطای تست برای طبقه‌بندهای V1 و V2

با توجه به شکل ۵.۱، تحلیل نتایج به شرح زیر است: طبقه‌بند V1 عملکرد به مراتب بهتری هم در زمینه کاهش خطای تست و هم در افزایش دقت نسبت به طبقه‌بند V2 از خود نشان می‌دهد. دلیلش به کیفیت استخراج ویژگی در انکدرها برمیگردد. انکدر V1 (با خروجی ۸ بعدی) توانسته است ویژگی‌های معنایی و متمایزکننده‌ی بهتری از ارقام را در فضای پنهان حفظ کند. در حالی که فضای پنهان ۴ بعدی در مدل V2 بیش از حد فشرده بوده و باعث از بین رفتن ویژگی‌های کلیدی شده است، که بعداً کار را برای طبقه‌بند سخت‌تر کرده است. تمامی این مقایسه‌ها عادلانه بوده است چون تعداد پارامترها در جفت معماری‌ها در حدود هم بوده است.

راهکارهای پیشنهادی برای بهبود عملکرد طبقه‌بند:

۱. فریز نکردن وزن‌های انکدر: تسکی که برای اتوانکدر تعریف کردیم بازسازی بود و نه طبقه‌بندی، لزوماً ویژگی‌های استخراج شده برای بازسازی بدرد طبقه‌بندی ممکن است نخورد. بخاطر همین میتوانیم وزن‌های انکدر را از حالت فریز دربیاریم (حداقل لایه‌های آخر).
۲. تغییر معماری به CNN: به صورت کلی، برای داده‌های عکس لایه‌های Fully Connected انتخاب خوبی نمیباشد چون که اطلاعات مکانی و ساختار دو بعدی پیکسل‌ها را نادیده میگیرند. در عوض شبکه‌های CNN با فیلترهای خود رابطه‌های محلی را در نظر میگیرند و طبقه‌بند قوی‌تری برای داده‌های عکس هستند.
۳. افزایش پیچیدگی شبکه‌ی طبقه‌بند: با اضافه کردن لایه‌های پنهان به شبکه، میتوانیم تعداد نرون

ها را زیادت‌کنیم و یعنی تعداد پارامترها نیز بیشتر میشود و مدل درجه آزادی بیشتری دارد و مرزهای تصمیم‌گیری پیچیده‌تری را پیدا میکند.

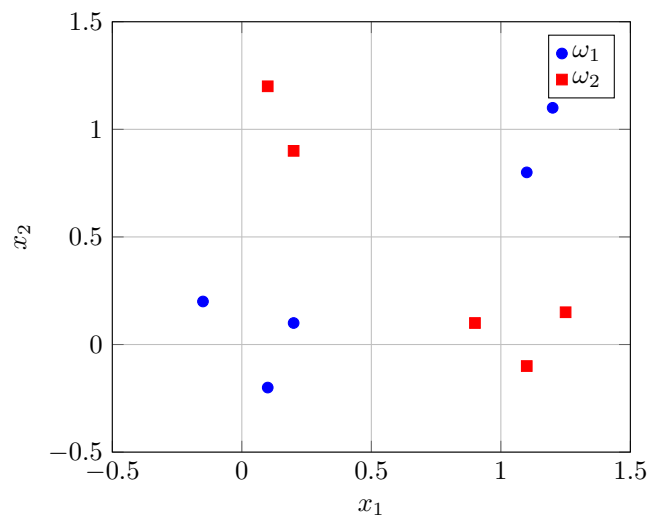
سوال ۲

Regression

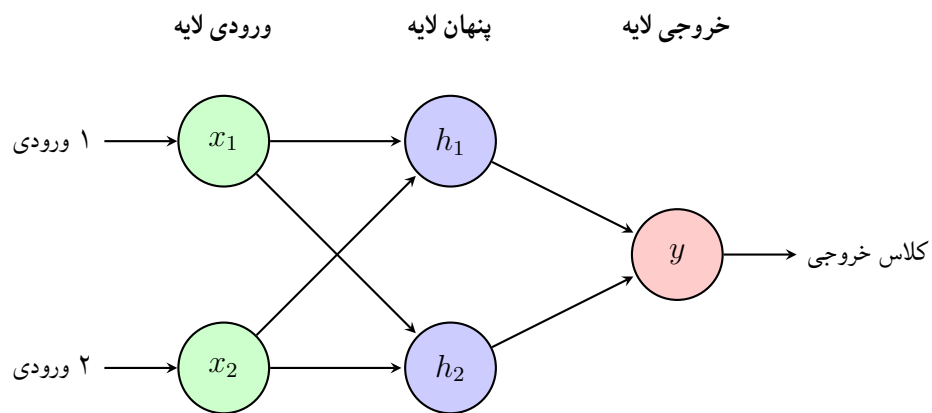
۱.۲ تئوری

۱.۱.۲ MLP

بردارهای ویژگی داده شده برای دو کلاس w_1 و w_2 را روی نمودار ۱.۲ می‌توانید مشاهده کنید. به وضوح معلوم است که این بردارها به صورت خطی تفکیک پذیر نیستند. از آنجا که داده‌ها خطی تفکیک پذیر نیستند، یک پرسپترون تک لایه نمی‌تواند آن‌ها را دسته‌بندی کند. اما با یک لایه پنهان با دو نورون می‌توانیم جدا کنیم دو کلاس را. دو ورودی می‌خواهیم چون بردار ویژگی مان ۲ بعدی است. ۲ نورون برای لایه پنهان می‌خواهیم. هر کدام از آنها یک مرز تصمیم‌گیری خطی درست می‌کنند و ترکیب این دو خط می‌تواند فضا را به ۴ ناحیه تقسیم کنند که با توجه به پخش داده‌ها که بالاتر دیدیم، می‌فهمیم با ۴ ناحیه می‌توان این دو کلاس را از هم جدا کرد. ۱ نورون هم برای لایه خروجی نیاز داریم چون طبقه بندی باینری می‌باشد. (می‌تواند برای کلاس w_1 مقدار 0 و برای کلاس w_2 مقدار 1 را تولید کند.) برای گره‌های لایه پنهان و لایه خروجی، باید از توابع فعال‌ساز غیرخطی استفاده کرد. مانند تابع پله یا سیگموئید یا ReLU. دلیل ضرورت این توابع این است که باید عنصر غیرخطی به شبکه مان اضافه کنیم. اگر فعال‌ساز مان خطی باشد اهمیت ندارد چند لایه پنهان داشته باشیم در نهایت می‌توان کل شبکه را به صورت $f(x) = (...(w_2^T.(w_1^T.x + b_1) + b_2)...) = w^T.x + b$ نشان داد و یعنی کل شبکه یک تبدیل خطی انجام می‌دهد و معادل یک پرسپترون ساده می‌باشد و برای اضافه کردن پیچیدگی باید از توابع فعال‌ساز غیرخطی استفاده بکنیم.



شکل ۱.۲: توزیع بردارهای ویژگی دو کلاس ω_1 و ω_2



شکل ۲.۲: ساختار شبکه عصبی MLP $(2 \rightarrow 2 \rightarrow 1)$

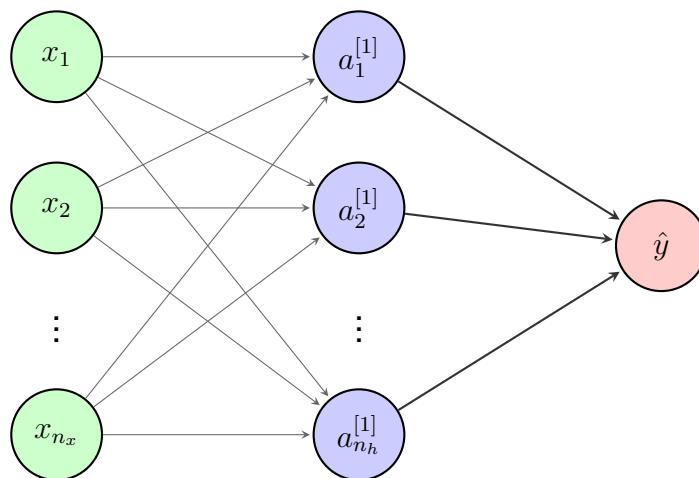
۲.۱.۲ Backpropagation

ساختار شبکه در شکل ۳.۲ آورده شده است.

الف. تعیین ابعاد ماتریس ها و پارامترها

رابطه لایه اول به صورت

$$Z_1 = W_1 X + b_1$$



شکل ۳.۲: ساختار شبکه عصبی دولایه با n_x ویژگی و n_h نورون پنهان

است. از آنجا که لایه پنهان n_h نورون دارد و خروجی آن برای m نمونه محاسبه می‌شود، ابعاد ماتریس Z_1 باید $n_h \times m$ باشد. برای برقراری ضرب ماتریسی $W_1 X$:

$$(n_h \times m) = (\text{ابعاد } W_1) \times (n_x \times m)$$

بنابراین، ابعاد ماتریس وزن W_1 باید $n_h \times n_x$ باشد. bias به همه نورون های لایه پنهان متصل شده است اما یک ویژگی بیشتر نیست (یعنی بجای n_x باید یک بگذاریم) و یعنی ابعادش $1 \times n_h$ میباشد. (توجه کنید که Broadcast میشود و مشکلی در صحت معادله ایجاد نمیکند.) ماتریس a_1 هم که فقط یک لایه فعال‌ساز را روی Z_1 اثر داده است و تاثیری روی ابعاد ندارد یعنی ابعاد آن $n_h \times m$ میباشد.

رابطه لایه خروجی به صورت

$$Z_2 = W_2 a_1 + b_2$$

است. جفت Z_2 و $\hat{Y}$ یک ابعاد دارند چون فقط تابع سیگموئید اثر داده شده است. هر دو یک عدد به ازای هر داده نگه میدارند و یعنی ابعاد آنها $1 \times m$ میباشد.

$$(1 \times m) = (\text{ابعاد } W_2) \times (n_h \times m)$$

بنابراین، ابعاد ماتریس وزن لایه خروجی W_2 باید $1 \times n_h$ باشد. بردار bias این لایه b_2 نیز تنها برای یک نورون خروجی است، پس یک اسکالر یا با ابعاد 1×1 است.

ب. محاسبه مشتق تابع هزینه نسبت به Z_2

$$J = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] = \frac{1}{m} \sum_{i=1}^m \left[L^{(i)} \right]$$

$$\frac{\partial L^{(i)}}{\partial z_2^{(i)}} = \frac{\partial L^{(i)}}{\partial \hat{y}^{(i)}} \cdot \frac{\partial \hat{y}^{(i)}}{\partial z_2^{(i)}}$$

$$\frac{\partial L^{(i)}}{\partial \hat{y}^{(i)}} = - \left(\frac{y^{(i)}}{\hat{y}^{(i)}} - \frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} \right) = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})}$$

$$\frac{\partial \hat{y}^{(i)}}{\partial z_2^{(i)}} = \sigma(z_2^{(i)})(1 - \sigma(z_2^{(i)})) = \hat{y}^{(i)}(1 - \hat{y}^{(i)})$$

$$\frac{\partial L^{(i)}}{\partial z_2^{(i)}} = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})} \cdot \hat{y}^{(i)}(1 - \hat{y}^{(i)}) = \hat{y}^{(i)} - y^{(i)}$$

$$\frac{\partial J}{\partial z_2^{(i)}} = \frac{1}{m} (\hat{y}^{(i)} - y^{(i)})$$

$$\frac{\partial J}{\partial Z_2} = \left[\frac{\partial J}{\partial z_2^{(1)}}, \frac{\partial J}{\partial z_2^{(2)}}, \dots, \frac{\partial J}{\partial z_2^{(m)}} \right]$$

$$\frac{\partial J}{\partial Z_2} = \left[\frac{1}{m} (\hat{y}^{(1)} - y^{(1)}), \frac{1}{m} (\hat{y}^{(2)} - y^{(2)}), \dots, \frac{1}{m} (\hat{y}^{(m)} - y^{(m)}) \right]$$

$$\frac{\partial J}{\partial Z_2} = \frac{1}{m} (\hat{Y} - Y)$$

۲.۲ پیاده سازی

۱.۲.۲ پیش پردازش داده ها

مجموعه داده مورد استفاده مربوط به امید به زندگی (Life Expectancy) است. در ابتدا، ستون نام کشور (country) به دلیل دسته‌بندی بودن و عدم وجود رابطه ترتیبی معنادار حذف شد تا از کاهش تعمیم‌پذیری مدل جلوگیری شود. همچنین متغیر وضعیت (status) با استفاده از Label Encoding به مقادیر عددی تبدیل شد. مراحل اصلی پیش‌پردازش به شرح زیر انجام شده است:

- مدیریت مقادیر گمشده و داده‌های پرت:

– مقادیر گمشده: ابتدا ردیف‌هایی که متغیر هدف (life_expectancy) در آن‌ها ناموجود بود حذف شدند. برای ویژگی‌ها، اگر درصد مقادیر گمشده ستونی در داده‌های آموزش بیشتر از 15% بود، آن ستون به طور کامل حذف گردید. برای مقادیر گمشده باقی‌مانده، از روش جایگذاری با میانگین استفاده شد (میانگین بر اساس داده‌های آموزش محاسبه شد). دلیل این کار حفظ ساختار داده بدون از دست دادن اطلاعات ارزشمند سایر ویژگی‌ها است.

– داده‌های پرت: برای شناسایی داده‌های پرت از روش Z-score با آستانه $Z \geq 3.0$ استفاده شد که فرمول آن به صورت

$$Z = \frac{|X - \mu|}{\sigma}$$

است. ستون‌های سال (year) و وضعیت (status) از این بررسی مستثنی شدند زیرا استفاده از این روش برای داده‌های زمانی و دسته‌بندی از نظر آماری منطقی نیست. در صورت صفر بودن انحراف معیار ($\sigma = 0$)، نمره صفر در نظر گرفته شد تا خطای تقسیم بر صفر رخ ندهد. دلیل استفاده از این روش، فرض توزیع نرمال و حذف حدود 1% از داده‌های انتهایی (نویزها) برای بهبود یادگیری مدل است.

• تقسیم داده‌ها: (Train/Val/Test) داده‌ها به سه بخش مستقل تقسیم شدند:

– داده‌های آموزش: (Train) 70% از کل داده‌ها

– داده‌های اعتبارسنجی: (Validation) 15% از کل داده‌ها

– داده‌های تست: (Test) 15% از کل داده‌ها

• مقیاس‌دهی (Scaling) داده‌ها: برای نرمال‌سازی داده‌ها از StandardScaler استفاده شد تا میانگین ویژگی‌ها برابر با 0 و انحراف معیار آن‌ها برابر با 1 شود ($\mu = 0, \sigma = 1$). پارامترهای مقیاس‌دهی صرفاً بر روی داده‌های آموزش Fit شدند و سپس برای تبدیل داده‌های آموزش، اعتبارسنجی و تست استفاده شدند تا از Data Leakage جلوگیری شود. همچنین متغیر هدف (y) نیز مقیاس‌دهی شد. دلیل این امر این است که بازه مقادیر هدف بین حدود 40 تا 90 است و شبکه‌های عصبی که وزن‌های آن‌ها در ابتدا با مقادیر کوچک مقداردهی می‌شوند، برای رسیدن به این بازه نیاز به گام‌های به‌روزرسانی بسیار زیادی دارند که همگرایی را کند می‌کند.

۲.۲.۲ آموزش شبکه عصبی ساده

معماری شبکه‌ها و تعداد پارامترها:

- مدل اول: این شبکه شامل یک لایه ورودی، یک لایه پنهان با 16 نورون و تابع فعال‌ساز ReLU، و یک لایه خروجی با یک نورون (خطی) است. تعداد کل پارامترهای قابل آموزش این مدل برابر با 305 پارامتر است.
- مدل دوم: این شبکه شامل یک لایه ورودی، دو لایه پنهان به ترتیب با 16 و 8 نورون همراه با توابع فعال‌ساز ReLU، و یک لایه خروجی با یک نورون (خطی) است. تعداد کل پارامترهای قابل آموزش این مدل برابر با 433 پارامتر است.

روند آموزش:

هر دو مدل با استفاده از تابع هزینه میانگین مربعات خطا (MSE) و بهینه‌ساز Adam با نرخ یادگیری 0.0001 به مدت 200 اپیاک آموزش داده شدند. در هر اپیاک، عملکرد مدل بر روی داده‌های آموزش و اعتبارسنجی محاسبه و ثبت گردید.

معیارهای ارزیابی:

برای ارزیابی نهایی بر روی داده‌های تست، از معیارهای زیر استفاده شد (که مقادیر پیش‌بینی شده به مقیاس اصلی برگردانده شده‌اند):

- MSE: میانگین مربعات خطا.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- RMSE: ریشه میانگین مربعات خطا.

$$RMSE = \sqrt{MSE}$$

- R^2 : ضریب تعیین.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

نتایج ارزیابی نهایی (داده‌های تست):

- مدل اول (یک لایه پنهان):

$$MSE = 13.3558 -$$

$$RMSE = 3.6546 -$$

$$R^2 = 0.8566 -$$

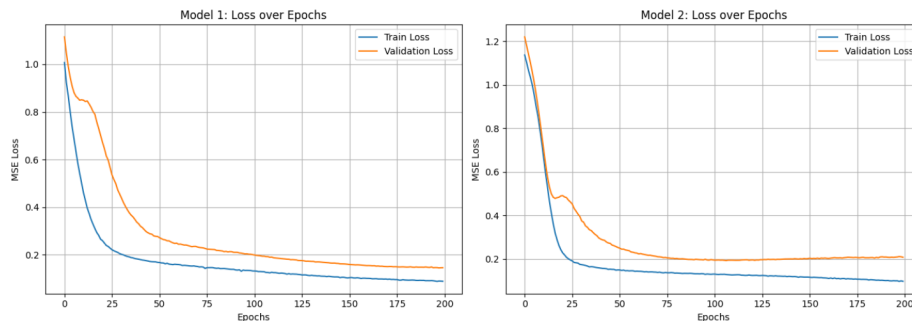
• مدل دوم (دو لایه پنهان):

$$MSE = 12.8045 -$$

$$RMSE = 3.5783 -$$

$$R^2 = 0.8625 -$$

همانطور که مشخص است، مدل دوم به دلیل داشتن عمق بیشتر و ظرفیت بالاتر (پارامترهای بیشتر)، توانسته است الگوهای پیچیده‌تری را یاد بگیرد و خطای کمتری (MSE پایین‌تر) و ضریب تعیین بهتری (R^2 بالاتر) روی داده‌های تست ثبت کند. (البته مدل دو نسبت به یک تعداد پارامترها را کمی زیاد تر میکند و مدل‌ها هنوز محدود هستند و در مدل پیشرفته که جلوتر معرفی میکنیم تقریباً ده برابر میکنیم این تعداد پارامترها را و نتایج کمی بیشتر به چشم می‌آیند). نمودارهای تغییرات Loss: نمودار تغییرات تابع هزینه (MSE Loss) برای داده‌های آموزش و اعتبارسنجی در طول ایپاک‌ها برای هر دو مدل در شکل زیر آورده شده است.



شکل ۴.۲: تغییرات تابع هزینه (Loss) در طول ۲۰۰ ایپاک برای مدل‌های اول و دوم

۳.۲.۲ آموزش شبکه عصبی پیشرفته

معماری شبکه و دلیل انتخاب آن:

در این مرحله، یک مدل شبکه عصبی پیشرفته‌تر (مدل سوم) طراحی شد. این مدل شامل لایه‌های پنهان بیشتر (به عنوان مثال ۳ لایه پنهان با تعداد نورون‌های بیشتر مانند ۶۴، ۳۲ و ۱۶)، توابع فعال‌ساز ReLU، و همچنین استفاده از تکنیک‌های منظم‌سازی (Regularization) است. تعداد کل پارامترهای قابل

آموزش این مدل برابر با 3697 پارامتر است. دلایل استفاده از این معماری به شرح زیر است:

- لایه‌ها و نورون‌های بیشتر: به شبکه اجازه می‌دهد تا الگوها و روابط غیرخطی پیچیده‌تری را بین ویژگی‌ها و امید به زندگی یاد بگیرد.
- استفاده از Batch Normalization: این لایه‌ها با نرمال‌سازی خروجی لایه‌های قبلی، به پایداری گرادین‌ها در طول آموزش کمک کرده و سرعت همگرایی مدل را افزایش می‌دهند.
- استفاده از Dropout: با توجه به افزایش تعداد پارامترها، خطر Overfitting افزایش می‌یابد. استفاده از لایه Dropout (با غیرفعال کردن تصادفی درصدی از نورون‌ها در هر مرحله آموزش) باعث می‌شود شبکه به ویژگی‌های خاص وابسته نشده و قدرت تعمیم‌پذیری آن روی داده‌های تست افزایش یابد.

نتایج ارزیابی نهایی:

مدل سوم نیز با همان شرایط (بهینه‌ساز Adam، تابع هزینه MSE) آموزش داده شد. نتایج ارزیابی روی داده‌های تست (پس از بازگرداندن به مقیاس اصلی) بهبود قابل توجهی را نسبت به دو مدل قبلی نشان می‌دهد:

$$MSE = 10.5086$$

$$RMSE = 3.2417$$

$$R^2 = 0.8872$$

همان‌طور که مشاهده می‌شود، ترکیب عمق بیشتر و تکنیک‌های منظم‌سازی باعث کاهش بیشتر خطا و افزایش قدرت پیش‌بینی مدل شده است.

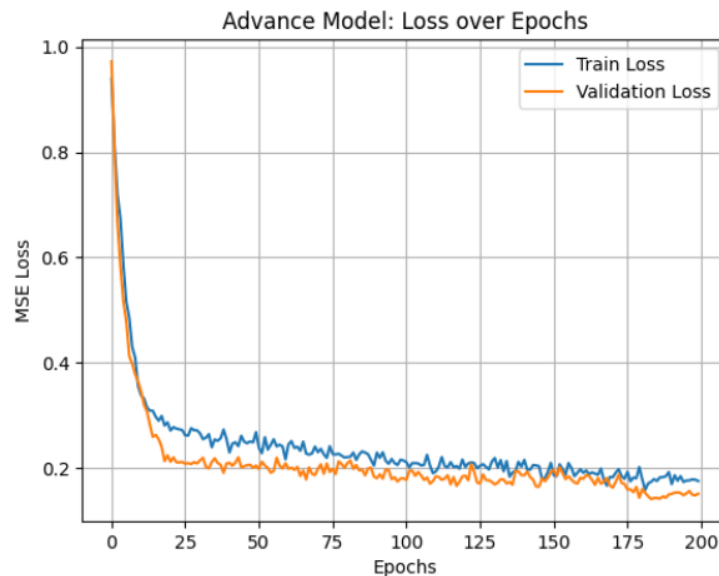
نمودار تغییرات Loss:

نمودار تغییرات تابع هزینه (MSE Loss) برای داده‌های آموزش و اعتبارسنجی در طول ایپاک‌ها برای مدل پیشرفته در شکل زیر آورده شده است. پایداری بهتر نمودار اعتبارسنجی نشان‌دهنده تأثیر مثبت Dropout و Batch Normalization است.

۴.۲.۲ تحلیل نتایج

تفسیر و معنای معیارهای ارزیابی

برای سنجش عملکرد مدل‌ها روی داده‌های تست، مقادیر پیش‌بینی شده به مقیاس اصلی خود (سال‌های امید به زندگی) بازگردانده شده‌اند. مقادیر به دست آمده برای هر معیار به شرح زیر تفسیر می‌شوند:



شکل ۵.۲: تغییرات تابع هزینه (Loss) در طول آموزش برای مدل پیشرفته (مدل سوم)

- **معیار MSE :** این معیار میانگین مجذور اختلاف بین مقادیر واقعی و پیش‌بینی شده را نشان می‌دهد. به دلیل به توان دو رسیدن خطاها، این معیار خطاهای بزرگتر را بیشتر جریمه می‌کند. کاهش این مقدار از حدود 13.35 در مدل اول به 10.50 در مدل پیشرفته، نشان‌دهنده کاهش چشمگیر خطاهای بزرگ در پیش‌بینی‌های مدل نهایی است.
- **معیار $RMSE$:** از آنجا که واحد MSE مجذور واحد اصلی داده‌هاست، از $RMSE$ استفاده می‌کنیم تا خطا به واحد اصلی (یعنی «سال») برگردد. مقدار $RMSE \approx 3.24$ در مدل پیشرفته به این معناست که پیش‌بینی‌های مدل ما برای امید به زندگی، به طور میانگین حدود 3.24 سال با مقدار واقعی اختلاف دارد. با توجه به بازه امید به زندگی (حدود ۴۰ تا ۹۰ سال)، این میزان خطا برای یک مدل رگرسیون قابل قبول و نشان‌دهنده دقت خوب آن است.
- **معیار R^2 :** این معیار نشان می‌دهد که چه کسری از واریانس متغیر هدف (امید به زندگی) توسط ویژگی‌های ورودی (متغیرهای مستقل) از طریق مدل ما توضیح داده شده است. مقدار $R^2 = 0.8872$ در مدل پیشرفته به این معناست که مدل توانسته است حدود 88.72% از تغییرات امید به زندگی در میان کشورهای مختلف را بر اساس داده‌های ورودی یاد بگیرد و پیش‌بینی کند. هرچه این مقدار به 1 نزدیک‌تر باشد، مدل قدرت تبیین بالاتری دارد.

مقایسه شبکه‌ها و نتیجه‌گیری

برای مقایسه بهتر، نتایج هر سه مدل در جدول زیر خلاصه شده است:

نام مدل	تعداد پارامترها	MSE	$RMSE$	R^2
مدل اول (یک لایه پنهان)	305	13.3558	3.6546	0.8566
مدل دوم (دو لایه پنهان)	433	12.8045	3.5783	0.8625
مدل سوم (پیشرفته)	3697	10.5086	3.2417	0.8872

جدول ۱.۲: مقایسه عملکرد سه معماری مختلف شبکه عصبی

۱. تأثیر عمق شبکه (مدل ۱ در برابر مدل ۲): با افزودن یک لایه پنهان دیگر (تغییر از ۱۶ نورون به معماری ۱۶-۸) و افزایش اندک تعداد پارامترها از 305 به 433، مدل توانست الگوهای غیرخطی‌تری را استخراج کند. این امر باعث بهبود R^2 از 0.8566 به 0.8625 و کاهش $RMSE$ شد. با این حال، ظرفیت محدود این دو مدل اجازه یادگیری ویژگی‌های بسیار پیچیده را نمی‌دهد.

۲. تأثیر افزایش ظرفیت و منظم‌سازی (مدل پیشرفته در برابر ۱ و ۲): در مدل سوم، تعداد پارامترها تقریباً ده برابر افزایش یافت (3697 پارامتر). به طور معمول، این افزایش شدید ظرفیت می‌تواند منجر به Overfitting شود. با این حال، همان‌طور که در نمودارهای تابع هزینه این مدل مشهود است، استفاده از تکنیک‌های منظم‌سازی Dropout و Batch Normalization توانست از این مشکل جلوگیری کند.

۳. جمع‌بندی: بهترین عملکرد متعلق به مدل پیشرفته است. این مدل با $R^2 \approx 88.7\%$ و خطای میانگین 3.24 سال ($RMSE$)، بهترین نتیجه را برای این مسئله رگرسیون داده است.